OmniSight Phase B: Summary

Sprint 1 (30 Aug – 5 Sep): sign-in and the empty app

We built the foundation: the database, the API service and the web app, all running in Docker behind HTTPS. A user signs in with an email and password (stored only as a secure hash) and gets a token that expires after 15 minutes without use. Every protected page and API route checks that token and the user's role, and an account locks after 5 wrong passwords. The automated tests passed: TC-10 15 of 15, NFR-9 2 of 2. Two things are still open: HTTPS (NFR-8) works but was never measured and recorded, and only the admin account exists, so the role and lockout checks were tested by the automated tests, not by hand.

How to check it

Start the system: in `phase-b`, run `docker compose up -d` and wait until every container is running (`docker compose ps`).

In the browser:

#	Do	Expected
1	Type `http://localhost`	It switches to `https://localhost` by itself
2	Look at the address bar	The lock icon (with a warning, because the certificate is self-made)
3	Go straight to `https://localhost/videos` without signing in	Sent to the sign-in page
4	Sign in with a wrong password	A clear error; still on the sign-in page
5	Sign in with the right admin password	The dashboard opens
6	Refresh the page (F5)	Still signed in
7	Look at the sidebar	Only the screens the admin can open, and each one opens
8	Sign out	Back at the sign-in page
9	Press the browser's Back button	Cannot get back into the app
10	Sign in, leave the tab untouched for 16 minutes, then click something	Sent back to sign in (15-minute expiry)

Not testable from the screen yet (only one user exists, and there is no screen to create others): the wrong-role block and the lockout after 5 wrong passwords. The automated tests cover both. Do not try the lockout on the admin: it locks the account for 15 minutes.

In the database (DBeaver: PostgreSQL, `localhost:5432`, database `omnisight`, user and password from `.env`), table `users`:

Column	What to see
`email`, `role`	The seeded admin, role `admin`
`password_hash`	A long scrambled string starting with `$argon2`, never the real password
`failed_attempts`	Goes up by one with each wrong password, back to 0 after a correct sign-in
`locked_until`	Empty normally; a time 15 minutes ahead after 5 wrong passwords
`last_login_at`	The time of the last successful sign-in

Sprint 2 (6 – 12 Sep): cameras and uploads

We added the two ways video enters the system. An admin registers a camera by its address and tests it: the system reports its resolution and frame rate, or a clear reason it cannot connect. A user uploads a video file: it is stored, measured (duration, frame rate, size) and listed in the Video Archive. A separate internal service is the only one that ever touches video. A simulator with ten looping camera feeds stands in for real cameras. We proved nothing is lost: uploads survive a restart and a full shutdown, and a crash in the middle of an upload leaves no broken record (NFR-5, 3 of 3). The sprint's demo passed 5 of 5 on a wiped system.

How to check it

Start the system: `docker compose up -d`, then sign in as admin.

In the browser:

#	Do	Expected
1	**Cameras** → add a camera, address `rtsp://mediamtx:8554/cam04`	It appears in the list
2	Click **Test Connection**	`320x240 @ 30 fps`, status `connected` (`cam01`–`cam03` give `1280x720 @ 23.97`)
3	Add a camera with a wrong address, e.g. `rtsp://mediamtx:8554/nothing`	Test reports a specific reason (`unreachable`)
4	**Video Archive** → upload a short MP4	A progress bar, then the video in the list with the right duration and size
5	Filter the archive by date	Only matching videos are shown
6	Run `docker compose restart`, then refresh	The video is still there and still plays

Not testable from the screen: the operator-cannot-delete check, because only the admin user exists.

In the database:

Table	What to see
`cameras`	Your camera: `rtsp_url`, `location`, `group_name`, `domain`, `status` (`connected` or `unreachable`), `last_seen_at`
`videos`	Your upload: `filename`, `size_bytes`, `duration_s`, `fps`, `width`, `height`, `status`, `retention_until` (30 days after the upload)

Sprint 3–4 (20 – 29 Sep): frames, motion and live capture

We built Algorithm 1 from Phase A: every frame is compared with the one half a second earlier on an 8x8 grid, and only frames with enough motion are kept, plus one per second so still scenes are not lost. It runs on uploaded files and on live cameras through the same code. A recording can be played with its kept frames under it, clicking a frame jumps to that second, and a slider re-runs the gate. A camera can be started and left running: it reconnects by itself and counts any frame it has to drop. A durable queue makes sure nothing is lost when a part crashes (NFR-5: no frame lost across consumer kills, a broker rebuild and worker restarts). Measured: a live frame is stored in under 50 ms (NFR-1, first half), a camera and a file run together correctly (TC-1, first half), and on fixed cameras the gate keeps 5–20% of frames. Two weak points: on dash-cam footage the

gate keeps 57%, because a moving camera looks like motion, and at that time this machine handled 8 cameras, not 10.

How to check it

Start the system: `docker compose up -d`, then sign in as admin.

In the browser:

#	Do	Expected
1	**Video Archive** → open a video you uploaded	The player, with a grid of frames under it, marked kept or discarded with a reason (motion or heartbeat)
2	Click a kept frame's thumbnail	The player jumps to that second
3	Move the **Motion threshold** slider right, then left	Fewer frames kept, then more come back
4	**Cameras** → open a camera → **Start**	Status `connected`; frames arrive in a rolling strip; the counters climb
5	Run `docker compose stop mediamtx`	Status changes to `unreachable` within a second
6	Run `docker compose start mediamtx`	Back to `connected` by itself within about 10–20 s
7	Click **Stop**	Frames stop arriving

In the database:

Table	What to see
`segments`	One row per frame for a file: `frame_index`, `timestamp_s`, `motion_score`, `admitted`, `admission_reason` (`motion` or `heartbeat`), `clip_start_s`/`clip_end_s`. `frame_path` and `thumb_path` are filled only on kept frames. For a camera, `camera_id` is set and `video_id` is empty
`camera_stats`	Per camera: `captured`, `dropped`, `last_frame_at`

Notes from the review

- **A live camera is shown as frames, not as video.** Each tile fetches a fresh image about once a second (D-14), so it looks like a slideshow. A live camera should play as continuous video, while the frame checking behind it stays exactly as it is.
- **Deleting a camera only disables it.** A camera that already has recordings or stored frames is switched to `disabled` instead of being removed, so nothing is left pointing at a missing camera; one with no frames is really deleted. The screen does not say so, so Delete looks broken, and the choice is recorded only in the sprint 2 context notes, not in `decisions.md`.

Sprint 5 (30 Sep – 4 Oct): the camera wall

We built the Live Monitoring screen: a wall of cameras in a grid, each tile showing whether the feed is live, frozen, offline or stopped, never just a black box. Any camera can be enlarged and rewound through its last few minutes. Behind it, live cameras now check every frame as Algorithm 1 says, live frames have their own queue so an upload cannot delay them (NFR-1: under 0.4 s even with a long upload), and capture comes back by itself after a restart. TC-1 is now complete: a live camera and an upload run together on the wall with only motion frames kept.

NFR-4 was measured honestly: with every frame checked, this 4-core laptop (Intel i7-1165G7, 4 cores / 8 threads, 16 GB) sustains 3 live cameras for 10 minutes, not Phase A's 10. Since the limit is the machine's CPU, NFR-4 was revised to "3 per machine of this class", with more cameras by adding machines (D-19). There is no limit of 3 in the code; a machine with more CPU cores runs more cameras.

How to check it

Start the system: `docker compose up -d`, sign in as admin, start 3 cameras (the number this laptop sustains).

In the browser:

#	Do	Expected
1	Open **Live Monitoring**	A 3x3 wall, each tile labelled with name and location, updating about once a second, marked **Live**
2	Change the layout to 4x4 and pin a camera top-left, then refresh	The layout and pin stay
3	Run `docker compose pause mediamtx`	Tiles turn **Frozen** in about 3 s, then **Offline**; the last frame is dimmed with its age
4	Run `docker compose unpause mediamtx`	Tiles return to **Live** by themselves
5	Stop one camera from its page	Its tile shows **Stopped**
6	Click a tile	Full screen; **Rewind** steps back through kept frames, with motion and heartbeat frames told apart
7	Run `docker compose restart ingestion`	Tiles go **Offline**, then back to **Live** in about 15–20 s with nobody pressing Start

In the database:

Table	What to see
`cameras`	`capture_enabled` is true for a started camera (that is what brings it back after a restart)
`camera_stats`	`gated` (frames checked), `admitted_motion`, `admitted_heartbeat`, `discarded`, `dropped`; `gated` = kept + discarded
`segments`	For a camera, only kept frames are stored, each with a reason

Notes from the review

- **Live view as video:** as noted in sprint 3–4, the wall shows a fresh still about once a second, not live video. A camera should play as continuous video, while the frame checking behind it stays as it is.
- **Recordings for the last 30 days (new feature):** today Rewind shows only kept frames, as stills, and only the last 1 to 10 minutes; frames the gate discarded are never stored, and no camera video is recorded. Wanted: each camera records continuously as video, the user picks a day from the last 30 and watches it back as video, and anything older is deleted automatically. Like the number of cameras, 30 days is a setting whose storage depends on the machine (about 22–43 GB per 1080p camera per day in H.264, half that in H.265); on this laptop it only has to be shown working, not filled for 30 days.
- **NFR-4 is a cut to a Phase A requirement** (10 cameras → 3 per machine of this class), backed by measurement and by the fixes tried (a process per camera, fewer threads, a faster grid calculation in sprint 6). To be discussed with the supervisor.

Sprint 6 (7 – 17 Oct): real footage, and the dashboard comes alive

The product now opens on an Operations Dashboard: cameras online, footage held, frames indexed, storage and the last 24 hours of activity, each number linking to the screen behind it. Behind the scenes we prepared the data the forecasting model needs. Every clip of the three datasets is listed in one manifest, split once into training, validation and held-out sets that can never change, and the lead-up windows before each event are cut and exported. UCF-Crime's training videos have no event times, so they were taken from a published annotation set (UCA) by a written rule. 60 windows were checked by eye (CCD 30 of 30, UCF-Crime 23 of 30), and the SM-3 target was fixed at 1.80 s from a rule written before measuring. Model measurements run on Colab's GPU; the product holds only the checked sample.

The main ideas, explained

The manifest. A master list: one database table (`dataset_manifest`) with one row for every clip of the three datasets (1,900 UCF-Crime, 4,500 CCD, 275 VIRAT). Each row says the clip's dataset and name, what happens in it (for example `Robbery` , or `Normal`), when the event starts and ends, where that time came from (official or UCA), which split it belongs to, and whether it is loaded into the product. Each dataset stores its labels in its own format; the manifest turns all three into one table, so everything after it reads from one place.

The split (fixed once with seed 605, committed in `datasets/splits/` with hashes):

Dataset	Train	Validation	Held-out (final test)	How it was cut
UCF-Crime (1,900)	1,368 (688 anomaly + 680 normal)	242 (122 + 120)	**290** (140 + 150)	Held-out = UCF-Crime's official test set; validation carved from its training list, balanced by class
CCD (4,500)	3,132 (1,032 crash + 2,100 normal)	675 (225 + 450)	**693** (243 + 450)	About 70 / 15 / 15; crash clips from the same YouTube video always stay on the same side
VIRAT (275)	–	–	–	Evaluation only (search quality, SM-1); nothing is trained on it

Train is what the model learns from; validation is used to tune it (for example the alert thresholds); held-out is touched once, at the end, for the final numbers (SM-2, SM-3, SM-4). CCD's crash split is uneven because keeping clips from one source video together forces whole groups to one side; otherwise the same crash could be in training and in testing, and the score would look better than it is.

The SM-3 target (1.80 s). SM-3 asks how early the system warns before a crash. Phase A said at least 15 seconds, which is impossible on our data: every CCD clip is 5 seconds long and the crash starts about 3–5 s in. So the target comes from the data. On the held-out crash clips the typical (median) footage before the crash is 3.60 s; the rule "warn at least halfway through that lead-up" gives **1.80 s**. The rule was written before measuring, so the number cannot be chosen to flatter the model. There is no model yet; it is measured in slice 10.

The window check. A window is the piece of video just before an event (for example the 30 s before a robbery); the forecaster learns from these what it looks like *before* an event happens. A window is useful only if it ends where the event starts and does not contain the event; otherwise the model learns to recognise events instead of predicting them. The check screen plays each

window with the event marked, and a person clicks Verified or Wrong. **CCD: 30 of 30 correct** (its labels mark the crash frame by frame). **UCF-Crime: 23 of 30**: official times 13 of 15, UCA times 10 of 15, mostly because the window ended too late and the event was already visible.

D-26: models on Colab, only 60 clips in the product. The plan was to load all three datasets (5,065 clips, 50 GB) into the product. Measured first: 20–34 hours on this laptop, and the AI models need a GPU the laptop does not have. Real users never see these datasets; they connect cameras and upload videos. The datasets exist only to measure the models, so that runs on Colab's GPU with the same code and the same split. The product keeps only the 60 hand-checked clips, so the check screen can play them (sprint 7 also adds 11 VIRAT videos, so search has real footage).

How to check it

Start the system: `docker compose up -d`, then sign in as admin.

In the browser:

#	Do	Expected
1	Land on the dashboard	Cameras online/offline, videos, segments, storage, a 24-hour activity chart; the alerts panel says alerts come in slice 9
2	Click each number	Each opens its screen (cameras → wall, videos → archive, and so on)
3	Start a camera, wait 30 s	The numbers move without refreshing
4	Open **Datasets**	CCD, UCF-Crime and VIRAT, with loaded / total, windows derived and verified
5	Open a dataset's windows → open one	The video plays, the lead-up window is shaded and the event is marked; Verified / Wrong buttons

In the database:

Table	What to see
`dataset_manifest`	Every clip: `dataset`, `clip_key`, `event_class`, `event_start_s`, `annotation_source` (official or UCA), `split`; `video_id` set only for the loaded clips
`event_windows`	Each window: start, end, length, class, split, and `verified` with who and when
`segment_hourly`	The dashboard's hourly counts
`videos`	Dataset clips with `source = dataset` and an empty `retention_until` (never deleted)

Notes from the review

- **The UCF-Crime training windows rely entirely on UCA.** The official times exist only for the held-out test videos, and UCA-timed windows were wrong 1 in 3 in the hand check (K-20). Must be fixed before the forecaster trains in slice 9.
- **The 30 s and 60 s windows are mostly the same footage on UCF-Crime.** Events start early in the videos, so 67% of the 30 s and 84% of the 60 s windows were shortened (median real length about 15 s). Phase A's 30 s against 60 s comparison needs a decision before slice 9.

Sprint 7–8 (18 Oct – 1 Nov): search by scene and by action

You can now type a description inside a video, or in the live camera viewer, and get the five best moments back in about a second; click one and the player jumps to that second. Every kept

frame gets two "fingerprints": a **scene** one (what the frame looks like, from SigLIP 2) and an **action** one (what happens in the second before it, from InternVideo2). A search compares the words with both and merges the two lists (Algorithm 3). Speed passes: results in under 3 s, and a new frame is searchable 3.35 s after capture when the encoding runs on a GPU. Quality was measured on 50 test questions: the right footage is in the top 5 for 76% of them (random picks would get 25%). Places are found well (92%); actions are not found better than by the scene fingerprint alone. Phase A's 85% target turned out to have no published evidence behind it, so these results are reported as measured, not as pass or fail (D-41).

The main ideas, explained

Two models, two jobs. SigLIP 2 looks at one frame, so it is good at "a blue car" (1152 numbers per frame). InternVideo2 looks at 8 frames over one second, so it should catch movement, like opening against closing a trunk (512 numbers per clip; Phase A said 768, D-33). On Colab's GPU a frame takes about 0.1 s and a clip about 0.3 s. On the laptop a frame takes about 9 s, so the laptop loads only InternVideo2's small text part, and the action fingerprints of our footage are made on Colab (D-32).

Qdrant. A database made for fingerprints. It holds two collections, `frame_vectors` (scene) and `clip_vectors` (action), and finds the closest ones to a question in milliseconds.

The test archive on Colab. All 275 VIRAT videos went through our gate on Colab: 793,686 frames checked, 35,704 kept, each with both fingerprints (2.5 hours on an A100). The laptop and Colab give the same fingerprints for the same frame.

The 50 test questions. Written by Claude Code and locked before any search ran:

- **25 action questions** from the 11 kinds of action VIRAT labels (opening a trunk, getting into a car, and so on), about 2 wordings each, including the pairs one frame cannot tell apart (opening against closing, in against out)
- **25 place questions**, written by looking at a picture of each of VIRAT's 11 places ("*a parking lot beside a brick building*")
- Every question has an answer in the videos: actions that never happen (digging) were left out

How a result is marked right. An action answer is right if its frame lies inside a labelled event of that action, in any video; a place answer is right if it comes from that place. The score is the share of questions with at least one right answer in the top 5.

Search	All	Places	Actions
Scene only	74%	88%	60%
Action only	46%	60%	32%
Both, merged (the product)	**76%**	**92%**	**60%**
Random picks	25%	39%	12%

What published research says (D-41). SigLIP 2 finds the right image first 56% of the time on COCO and 86% on Flickr30k; InternVideo2 (1B) finds the right video first 52% of the time on MSR-VTT. Our smaller InternVideo2 has no published number, and nobody has published search results on surveillance footage like VIRAT. Research also shows that most video models barely use motion (a single frame does as well on popular tests) and are near chance at telling a video played forward from backward. So 76%, with no gain on actions, is in line with what is known.

Results

Requirement	Result
NFR-2, results under 3 s	**Pass**: slowest 2.02 s on the laptop (1,800 searches), 0.22 s with a GPU
NFR-1, searchable under 5 s	**Pass** with the GPU's encode time: slowest 3.35 s (270 s on the laptop's CPU)
SM-1, search quality	**Measured** (D-41): 76%, 3 times chance; no gain on actions
TC-4, a large archive	**Measured** (D-41): 74% on 35,704 frames (not 100,000, D-37)
TC-2, TC-3	**Pass** (cameras get no action fingerprint on the laptop)

How to check it

Start the system: `docker compose up -d`, then open **https://localhost** (not port 3000) and sign in. The first search after a pause can take about 10 s (K-26).

#	Do	Expected
1	Videos → open a VIRAT video → type *a blue car* → Search	5 pictures, each with its time, score, a badge (Scene, Action or both) and the time taken
2	Click a result	The player jumps to that second
3	Open video 3231 → **Compare** → *a person unloading an object from a car*	Three columns side by side; Scene and Combined find it at 104–111 s, Action misses
4	Open a live camera → search	Results open a still frame, not video (K-29)

The full five-minute demo for the supervisor is `docs/demo.md`.

Notes from the review

- **SM-1 and TC-4 are reported as measured, not failed (D-41).** Phase A's 85% was our own guess, with no published result behind it.
- **SM-1 was run twice.** Algorithm 3 as written got 60% (it sent action questions to the weaker action list); with weights chosen on 50 other questions it got 76% (D-40). Both are recorded.
- **Nobody has checked the 50 questions.** Claude Code wrote them; reading them once before the meeting makes them easier to defend. An answer counts by its time, even if the action is not clearly visible in that frame, and "carrying" is found 90% of the time by chance alone.
- **There is no separate Search screen (D-39).** Search lives inside each video and the live viewer, and searches that footage only.
- **The 60 hand-check clips have no fingerprints (D-35):** about 77 hours on the laptop.